
SYNTACTIC DIVERSITY IS NOT BEHAVIORAL DIVERSITY: MEASURING AND REPAIRING COLLAPSE IN AI- GENERATED HARNESS POPULATIONS

Anonymous

ABSTRACT

Large language models are increasingly wrapped in *harnesses*—generated programs of prompts, tools, and control flow—and a growing literature routes over *populations* of such harnesses per query. Routing presupposes that population members disagree behaviorally. We show this presupposition fails for automatically generated populations: on BIRD text-to-SQL with a frozen mid-tier target, the candidates of a standard harness-evolution loop are syntactically diverse (mean pairwise code similarity 0.14) yet behaviorally collapsed (96% outcome agreement; 21 of 66 candidate pairs identical on every task; the shipped ReAct variant reproduces `bare`’s final SQL byte-for-byte on every audited task). Execution-trace auditing resolves the collapse into three failure classes: mechanisms described but absent from the control flow, feedback with no causal path to the output, and broken multi-turn paths. A hand-written positive control on the same target, tasks, and judge delivers 7.3 pp oracle headroom on 151 held-out questions—the headroom routers need is real, and it is the *generation* side that fails. We then probe both axes with a strategy-forced free-form protocol: two of three builders produce populations that exceed pre-specified diversity thresholds at the point estimate for the first time (6.0 and 8.0 pp headroom; six pairwise-distinct repair sets each), the old un-gated protocol misses, and under fully logged generation the builder axis orders per-strategy acceptance (GLM 3/8, Qwen 7/8, DeepSeek 8/8). The design cannot separate protocol and builder effects. Diversity, however, is necessary, not sufficient: a leave-one-harness-out value predictor over the admitted populations does not beat the best fixed harness (≤ 0.001 AUROC over task-only features), and in stratified task-held-out controls its repair signal is at chance; its one deployment-relevant success—+6.9 pp over `bare` with zero induced harm—holds only as an abstaining gate over the full 24-harness pool. We release a three-metric precondition checklist—pairwise disagreement, union repair, oracle headroom—that harness-generation and harness-routing papers can and should report before routing is attempted.

1 INTRODUCTION

The fastest-growing lever on a frozen LLM is no longer weights but the *harness* — the program of prompts, tools, and control flow that wraps the model for a task. A recent wave of systems generates this scaffolding automatically: LLM builders propose, edit, and evolve harness code, with reported gains over `bare` prompting. The implicit premise of this “AI4AI” line is that when the generated code differs, the *behavior* differs — that a population of thirty harnesses is thirty ways of answering, not one way written thirty times.

We test that premise directly. On BIRD text-to-SQL with a frozen mid-tier target, we take the full candidate population produced by a standard harness-evolution loop and measure what its members actually do, not what their source files say. Every candidate passes a code-difference check, yet among the 12 generated candidates average pairwise outcome disagreement is **3.8%** and **21 of**

66 candidate pairs are behaviorally identical (28 of 78 pairs once the shipped ReAct harness is included); the ReAct variant reproduces the bare baseline’s final SQL **byte-for-byte on every audited task**. Execution traces resolve “different code” into a three-class taxonomy — mechanisms *described* but absent from the control flow, feedback with no causal path to the output, and genuinely multi-turn paths that are broken. We call this **behavioral collapse**: syntactic diversity without behavioral diversity (F1).

Collapse is not a property of the task. Four hand-written harnesses with forcibly distinct execution paths — execute-and-repair, three-way voting, schema-linking, hint-first — run on the same frozen target and deliver a **6.7 pp oracle headroom** over the best fixed harness on 60 tasks (7.3 pp on 151), with three of the four producing repairs the others miss (F2). The headroom that harness routers and value predictors need is real; it is the generation side that fails to populate it.

If the bottleneck is generation, the question is whether it is a *capability* bottleneck (a stronger builder would fix it) or a *protocol* bottleneck (no builder is asked, checked, or forced to change behavior). We probe both axes with two nested contrasts (not a full factorial; Section 3.4): a pre-specified builder sweep under a **strategy-forced free-form** protocol — which assigns each candidate an explicit control-flow mechanism and accepts the emitted artifact only after an import and execution smoke check — run with GLM, Qwen, and DeepSeek; and a **Harness IR** that compiles declarative mechanism specifications into harness code deterministically, guaranteeing mechanism diversity by construction. On the builder-held-out 151-question set, both stronger-builder gated populations exceed all three pre-specified diversity thresholds at the point estimate — 5.96 and 7.95 pp oracle headroom, six pairwise-distinct repair sets each — while under the old protocol *every* tested builder fails the checklist: with Qwen and DeepSeek builders the optimize loop’s 40 candidates are behaviorally identical to one another (780/780 pairs, no repairs). In this design the protocol variable is load-bearing for behavioral diversity; builder identity governs generation reliability (3/8, 7/8, 8/8 strategies accepted, fully logged) and repaired-population quality (up to 9.93 pp). A K-controlled subsampling analysis rules out population-size effects (the old protocol never crosses the 5 pp line at any K; the DeepSeek repaired population exceeds it at K=2), and the repaired population’s checklist metrics replicate on a second frozen target (F3). A leave-one-harness-out value predictor over these admitted populations, however, does not beat the best fixed harness — and its apparent per-task signal is consistent with task memorization: in stratified task-held-out controls on D6 (five seeds, task text + harness identity features) its repair AUROC is at chance (mean 0.512). Behavioral diversity is *necessary* for routing value to exist, but estimating an unseen harness’s value from code and task text alone remains open (F4).

Our contributions:

1. **The first systematic outcome-matrix measurement of behavioral collapse** in AI-generated harness populations — the first measurement, on a full generated population’s outcome matrix, of the gap between syntactic diversity and behavioral diversity, with an execution-trace taxonomy explaining *why* code diversity fails to become behavioral diversity. Concurrent work uses behavior-aware verification inside evolution systems (Section 2); none measures population-level collapse as an outcome phenomenon.
2. **A controlled generation-side study**: two nested contrasts — a builder sweep (GLM / Qwen / DeepSeek) under the strategy-forced protocol plus an old-protocol arm and a compiled IR arm with a human positive control — evaluated under pre-specified admission thresholds on the builder-held-out 151-question set, showing behavioral diversity is restorable (two admitted populations), that the old protocol’s failure mode is quality concentration rather than absence of repairs, and that per-strategy generation acceptance orders with builder capability (3/8 vs 7/8 vs 8/8, logged).
3. **A boundary result for the routing literature**: on the admitted populations, open-set harness-value prediction fails to beat the best fixed harness (≤ 0.001 AUROC over task-only features), and in a task-held-out control its repair signal falls to chance — while the same predictor used as an abstaining gate over the 24-harness pool gains +6.9 pp over bare with zero induced harm there. Diversity is necessary, not sufficient, and the precondition checklist tells you which regime you are in.

Beyond harnesses, the failure pattern we isolate — LLM builders producing plausible, well-documented, behaviorally inert variants — is a generation-reliability phenomenon likely to recur wherever a model is asked to modify its own program.

2 RELATED WORK

2.1 HARNESS GENERATION FOR LLMs

A growing line of work generates task-specific scaffolding — prompts, tools, control flow — around a frozen model. TTHe evolves text-to-SQL harnesses with an LLM proposer/judge loop (TTHe authors, 2026); a-evolve and related evolutionary systems maintain and edit agent code populations; Self-Harness (Zhang, 2026) shows a model can improve its own harness; JIT-Agent (JIT-Agent authors, 2026) trains a model to *generate* a harness per task at inference time; Task-CoEvolve (Task-CoEvolve authors, 2026) co-evolves tasks and scaffolds. A concurrent 2026 cluster makes harness evolution *behavior-aware* from the verification side: HarnessLens (HarnessLens authors, 2026) allocates rollout budget and behavior-level checks during evolution; HarnessFix (Chen, 2026) repairs harnesses from trace-grounded diagnoses; AHE (Lin et al., 2026) verifies each proposed edit against self-declared behavioral predictions; gated semantic quality-diversity search (Gated-QD authors, 2026) evolves populations over LLM-assigned pathology descriptors; Harness Handbook (Harness Handbook authors, 2026) and Meta-Harness attack behavior localization and outer-loop search. These systems treat behavior as a *signal inside the loop*; none measures whether a generated population is behaviorally diverse as an outcome property, which is precisely the failure we quantify. Our strategy-forced free-form and harness-IR protocols are generation-protocol fixes, orthogonal to and composable with the search loops above.

2.2 HARNESS CONDITIONAL UTILITY AND ROUTING

Several recent papers establish that harness quality is conditional: “No Universally Superior Harness” (NUSH authors, 2026) and the non-monotonicity analysis of harness sensitivity (HSNM authors, 2026) document that no single harness dominates across instances. HELIX (HELIX authors, 2026) shows a portfolio of sibling harnesses beats the best fixed one and lists a harness *router* as future work; STS (STS authors, 2026) routes over six fixed human-designed scaffolds in a closed set; GRASP (GRASP authors, 2026) gates skill edits into a library at development time; CF search routing (CF-Routing authors, 2026) learns counterfactual, instance-level routing over {no-search, search, abstain}; agentic routing (Agentic Routing authors, 2026) routes *models* within a harness. What all of these consume — and none of them measure — is a population whose members actually disagree. We supply the missing precondition analysis, show it fails for naively generated populations, and give generation protocols that restore it.

2.3 BEHAVIORAL DIVERSITY IN LLM POPULATIONS

Diversity of LLM outputs has mostly been studied at the sample level (self-consistency, diverse decoding) or the agent-policy level. The observation that *syntactically distinct programs exhibit identical behavior* echoes coverage gaps documented in program synthesis and mutation-testing practice; our contribution is to instantiate it for AI-generated harness populations. To our knowledge (literature scan 2026-09-02, to be re-run at camera-ready), no prior work measures this collapse as an outcome property of a generated population; behavior-aware verification *inside* evolution systems is concurrent independent work (Section 2.1).

3 SETUP

3.1 HARNESSES AND HARNESS VALUE

Following the harness-abstraction of TTHe, a **harness** H is a program that wraps a frozen target model M : it receives the natural-language task input x , issues one or more calls to M , optionally executes intermediate artifacts (e.g., SQL against the database), and returns a final answer. Execution accuracy is scored programmatically. For a task x , we define the **harness marginal value**

$$\Delta(x, H, M) = Y(x, H, M) - Y(x, H_0, M) \in \{-1, 0, +1\},$$

where H_0 is the bare single-call baseline and $Y \in \{0, 1\}$ is programmatic correctness; $+1$ is a *repair* (raw wrong, harness correct), -1 is *harness-induced harm*. The deployment question that motivates this paper — predict Δ before executing H — presupposes that Δ is not almost-always zero. A

population of harnesses in which every harness reproduces the raw model’s outcomes offers nothing to predict or route over. Our first contribution is to take this presupposition seriously and measure it.

3.2 DOMAIN, TARGET, AND DATA SPLITS

We instantiate the problem on **BIRD text-to-SQL** (SQLite), whose answers are judged by execution against gold query results — no model-based grading. The target model is frozen throughout: **GLM-5.3-Flash** served through an OpenAI-compatible endpoint (temperature 0); builders differ only in how harness *code* is produced. The **builder-visible set** D_{build} contains 8 `card_games` questions; the **evaluation set** D_{eval} contains **151 questions** (121 `card_games`, 30 `formula_1`), held out from all builders. We disclose two overlaps explicitly: (i) 24 of the 151 items were also used in day-1 protocol-development measurements (their day-1 outcomes informed the diagnosis of Section 4, not any generation input); (ii) the four human control harnesses (Section 5) were designed after inspecting day-1 traces on these tasks, so the control’s headroom is an existence proof rather than a blind estimate. Populations are generated, frozen, and hashed *before* any evaluation on D_{eval} .

3.3 POPULATIONS UNDER COMPARISON

We compare seed baselines (`bare`; `react`, shipped with TTHE), four AI-generated populations (each $K = 6$: condition A, the old proposer protocol; conditions B/C/D, strategy-forced free-form with GLM / Qwen / DeepSeek builders), a compiled IR arm, and the human positive control (four hand-written harnesses with forcibly distinct execution paths). The old-protocol population A is selected for behavioral diversity using *only* the 36 earlier-run questions disjoint from D_{eval} , by a selection rule frozen before the 151-item evaluation; condition B is the audited logged re-run (B3, $K=3$), the original unaudited 09-01 run being disclosed as a provenance caveat (Section 6, Appendix B). Every harness is accepted only after passing a smoke check (it runs and emits executable SQL); the free-form arm additionally passes the execution-path gate of Section 6. Full condition definitions appear in Appendix B.

3.4 DESIGN STRUCTURE AND PRE-SPECIFIED DECISION CRITERIA

The design is two nested contrasts (the old-protocol row was later completed for Qwen and DeepSeek; Section 6.2): the protocol contrast at a fixed builder exists only for GLM (A vs B), the builder contrast exists only under the gated protocol (B vs C vs D), and the IR arm has a protocol axis but no builder axis. Section 6.2 completes the old-protocol row for the Qwen and DeepSeek builders; we interpret each contrast within its own fixed factor and do not claim to separate main effects. Before running any 151-item evaluation, admission thresholds were frozen in the project decision log (union repair over bare errors $\geq 15\%$, oracle headroom ≥ 5 pp, ≥ 3 harnesses with pairwise-distinct non-empty repair sets), with the v6/v7 audit trail documented in Appendix C; admission is decided on point estimates by the frozen rule, and Section 6 marks populations whose bootstrap intervals include sub-threshold values.

4 MEASURING BEHAVIORAL DIVERSITY

4.1 THREE METRICS

If harness quality is conditional — $Q(H \mid x, M)$ rather than a scalar $Q(H)$ — then a harness *population* is valuable only insofar as its members disagree on outcomes in a structured, repair-bearing way: code-level difference is cheap to obtain, but outcome-level difference is what routing consumes. We propose that any harness-generation pipeline (or any harness-routing paper) report three precondition metrics on a held-out set:

1. **Pairwise outcome disagreement.** Over all harness pairs (H_i, H_j) , the fraction of tasks where $Y(x, H_i, M) \neq Y(x, H_j, M)$. Zero disagreement means the population is behaviorally one harness.
2. **Union repair rate.** The fraction of bare errors — tasks where $Y(x, H_0, M) = 0$ — such that *some* harness in the population answers correctly. This bounds what any selector over the population could achieve.

3. **Oracle headroom.** Accuracy of the per-item oracle selector $\max_i Y(x, H_i, M)$ (taken over harnesses *and* the bare baseline) minus the best fixed harness. Positive headroom means the population’s value is *conditional on the item* — exactly the signal a value predictor or router would need.

These three metrics are cheap (they require only the outcome matrix $Y \in \{0, 1\}^{|X| \times |K|}$ that any evaluation already produces), hard to game (they are defined on outcomes, not on code), and diagnostic in combination: high disagreement with no union repair is noise; union repair with no headroom means one harness dominates; headroom without disagreement is impossible by definition.

4.2 TRACE AUDIT: WHY CODE DIVERSITY FAILS TO BECOME BEHAVIORAL DIVERSITY

Outcome metrics say *that* a population collapsed; they do not say *why*. We audit execution traces — the sequence of LLM calls and artifact executions per task — and find that “different code” decomposes into three failure classes:

- **T1 — Pure prompt variants (6/12 candidates).** The harness issues exactly one LLM call and never executes an artifact — the identical execution path as `bare`. The docstring may claim retrieval, self-checking, or token caps; none of it exists in the control flow. The mechanism is *described*, not *implemented*.
- **T2 — Feedback that cannot change the answer (react, 10/10 audited tasks).** The harness executes the SQL and feeds errors back to the model, yet its final SQL is byte-identical to `bare`’s on every audited task. Execution feedback exists in the loop but has no causal path to the output.
- **T3 — Buggy multi-turn paths (1/12).** The only candidate implementing genuine multi-turn repair returns prose as final SQL, converting bare-correct items into harness-wrong — the single negative-value behavior in the population.

This taxonomy turns “AI-generated harnesses are behaviorally collapsed” into an actionable diagnosis: T1 demands an execution-path acceptance gate (Section 6), T2 demands feedback mechanisms with a verified causal path, T3 demands executable-output validation — all three are *generation-protocol* requirements, independent of the benchmark.

4.3 THE COLLAPSE, QUANTIFIED

On the day-1 population (14 harnesses $\times$ 60 tasks, GLM-5.3-Flash target, TTTE proposer protocol): average pairwise outcome disagreement **3.5%** (maximum 13.3%; **28 of 78 pairs fully identical**), union repair over bare errors **7.7%** (2/26), oracle headroom **3.3 pp** — and `react` differs from `bare` on **zero** tasks. Every candidate passed a code-similarity check (“all files distinct, non-trivial diffs”): the population is syntactically diverse and behaviorally one harness. To our knowledge this is the first systematic measurement of this gap in AI-generated harness populations.

5 POSITIVE CONTROL: THE HEADROOM EXISTS

Before attributing collapse to generation, we must exclude the alternative: that BIRD/GLM simply has no exploitable instance-conditional signal — that *any* population, however diverse, could only be noise. We therefore wrote four harnesses by hand, each with an execution path forcibly different from bare:

- `hpc_repair` — execute the SQL; on error, feed the message back for one repair round;
- `hpc_vote3` — three independent samples at temperature > 0 , majority vote on execution results;
- `hpc_schema` — a schema-linking pass that prunes the schema before generation;
- `hpc_hint` — re-order the question into hint-first form before generation.

Same frozen target, same 60 tasks, same judge as the collapsed AI population. One auditable caveat: the bare baseline itself flipped 3 of 60 items between the two day-1 runs (26 vs. 29 bare errors; API

nondeterminism at temperature 0), so the columns are not conditioned on an identical bare-error set. On the 60-item run:

Metric	AI population (A)	Human positive control
Pairwise disagreement	3.5%	8.6%
Union repair of bare errors	7.7% (2/26)	20.7% (6/29)
Oracle headroom vs best fixed	3.3 pp	6.67 pp
Harnesses with distinct non-empty fix sets	1	3 (repair {36,38,53}, schema {11,12,38}, hint {38,43,53})
Union regressions	7	4

The fix-set audit (`hpc_fixset_audit.json`) makes the complementarity claim precise: re-pair/schema/hint each produce repairs the others miss (1/2/1 repairs unique to one harness), their fix sets are pairwise distinct, and `vote3` contributes no repair on this run. On the 151-item set the control’s headroom holds at **7.28 pp** [3.3, 10.0] while its repair rate settles at 14.9% — the 60-item 20.7% was small-sample optimism, disclosed in Section 7 — so two conclusions stand with tightened numbers: **F2** (the domain admits ≥ 5 pp of instance-conditional headroom at the point estimate) and the localization of the failure to the generation side.

6 BEHAVIOR-AWARE GENERATION PROTOCOLS

The diagnosis of Section 4 suggests two orthogonal fixes. **Strategy-forced free-form** keeps the builder fully expressive but forces an explicit mechanism per candidate and accepts the emitted artifact only after an import and execution smoke check; **Harness IR** compiles declarative mechanism specifications into harness code deterministically, so a declared mechanism is present in the control flow *by construction*. Builders see D_{build} only; no evaluation item or outcome is shown to any builder. Prompts, specs, acceptance rules, and per-attempt logs are specified in Appendix E and released verbatim. We compare six populations under identical budget and target — old proposer protocol (A), strategy-forced free-form with GLM (B) / Qwen (C) / DeepSeek (D), compiled IR, and the human control — on the builder-held-out 151-question set. Admission to any downstream routing claim requires the pre-specified thresholds of Section 3.4.

6.1 RESULTS TABLE (EVAL151, N=74 BARE ERRORS; CIs ARE TASK-LEVEL BOOTSTRAP 95%)

Population	disagree	union repair [CI95]	headroom pp [CI95]	distinct fix sets	admitted?
A old-proposer (K=6)	7.0%	24.3% [.149, .347]	4.64 [1.3, 8.0]	5	no (headroom)
B GLM free-form (logged re-run, K=3)	3/8 strategies accepted	see Section 6.0b	see Section 6.0b	3	see text
C Qwen free-form (K=6)	11.1%	21.6% [.125, .310]	5.96 [2.0, 8.6]	6	yes
D DeepSeek free-form (K=6)	11.2%	24.3% [.148, .346]	7.95 [4.0, 11.3]	6	yes
IR compiled (K=6)	8.2%	16.2% [.083, .253]	3.97 [1.3, 7.3]	6	no (headroom)
Human control (K=4)	8.9%	14.9% [.071, .237]	7.28 [3.3, 10.0]	4	no (repair, by 0.1pp)

Population	disagree	union repair [CI95]	headroom pp [CI95]	distinct fix sets	admitted?
Cross-protocol pool (K=24)	10.3%	37.8% [.268,.488]	11.26 [6.6,15.9]	23	yes (analysis)
B3 GLM logged re-run (K=3)	8.8%	21.6%	5.96	3	yes (post-hoc, K=3)
C8 Qwen completed grid (K=7)	10.4%	21.6%	5.96	7	yes (post-hoc)
D8 DeepSeek completed grid (K=8)	10.6%	28.4%	9.93	8	yes (post-hoc)

Rows 7–9 are the post-hoc completed-grid analysis (Section 6.2): the original admission decisions (rows 1–6) are unchanged; the GLM re-run population clears the checklist at K=3, though its small size limits any downstream claim.

Three observations. **(i) Behavior-aware generation restores behavioral diversity.** Both free-form populations built by Qwen and DeepSeek exceed all three pre-specified thresholds *at the point estimate* — the first AI-generated populations to do so — with disagreement more than 3× the day-1 level and six pairwise-distinct repair sets apiece. For calibration: the headroom intervals [2.0, 8.6] and [4.0, 11.3] include sub-threshold values; admission follows the frozen point-estimate rule, and the intervals are reported so readers can apply a stricter reading. **(ii) The contrasts localize the failure modes.** Within the old proposer protocol, the best member dominates (0.583, the single highest harness in the study), squeezing headroom to 4.64 pp despite a healthy repair rate — diversity without *spread*; this population misses the headroom threshold by 0.36 pp rather than collapsing outright (the outright collapse is the day-1 finding of Section 4). Under the logged generation tool, per-strategy acceptance orders with builder capability — GLM 3/8, Qwen 7/8 (its vote3 attempts end with well-formed prose but no code fence), DeepSeek 8/8 — replacing the unaudited original 0/6 observation (Appendix A discloses the provenance gap). Protocol and builder are jointly implicated by these two contrasts, though the design cannot separate their main effects (Section 3.4). **(iii) The human control is not a universal upper bound.** Its 60-item repair rate (20.7%) shrank to 14.9% on 151 items — small-sample optimism we had flagged in advance — while its headroom (7.28 pp) stayed strong: hand-designed harnesses are complementary specialists, not uniformly stronger solvers. We report the borderline cases (IR at 3.97 pp, human at 14.9%) exactly as measured, without post-hoc threshold adjustment.

6.2 COMPLETED-GRID RE-ANALYSIS (POST-HOC)

After the pre-specified comparison, we completed the strategy grid: DeepSeek gained schema-link and hint-guard (its weakest strategies are none; it accepts 8/8), Qwen gained execute-repair (its vote3 attempts consistently emit prose without a code fence and are logged; 7/8), and GLM was re-run under the logging tool (3/8). On the completed grids, every builder’s population clears the checklist: B3 (the three harnesses GLM produced) reaches 5.96 pp headroom and 21.6% union repair with three distinct fix sets; C8 holds at 5.96 pp / 21.6% with seven; D8 rises to **9.93 pp / 28.4%** with all eight strategies contributing a distinct fix set. Per-strategy accuracies show no builder dominance (Qwen wins hint-guard 0.556 vs 0.510; DeepSeek wins repair 0.543 vs 0.550 for Qwen; GLM’s vote3 reaches 0.543) — capability orders *generation reliability*, not per-strategy solution quality. These rows are labeled post-hoc: the frozen admission decisions belong to rows 1–6.

6.3 K-CONTROLLED HEADROOM CURVES (POPULATION-SIZE CONTROL)

Oracle headroom grows with K by construction, so cross-population comparison is only meaningful at matched K. Subsampling every population at $K = 2..K_{\max}$ (60 random subsamples per K, 95% bands in `kcurve.json / fig_kcurve.png`): at K=6 the old proposer protocol reaches 4.64 pp and never crosses the 5 pp line at any K, while the strategy-forced populations exceed it already at

K=2 (DeepSeek 6.08 pp; Qwen crosses only at K=4, 5.17 pp) and at every matched K thereafter (K=6: 8.92 / 5.68 pp). The protocol effect is not a population-size effect.

6.4 BUILDER X PROTOCOL: COMPLETING THE FACTORIAL

The old proposer protocol is builder-independent in its failure. We ran TTHER’s own optimize loop (the old protocol) with Qwen3.8-Flash and DeepSeek-V4-Flash builders (20 candidates each, GLM target, same budget), evaluated all 40 candidates on the 36 earlier-run items disjoint from Eval151: **all 780 candidate pairs are behaviorally identical** (mean disagreement 0.0%) and **no bare error is repaired by any candidate**. Combined with the day-1 GLM run (3.5% disagreement) and the strategy-forced arm (all three builders admitted), the completed design reads: under the old protocol every builder collapses; under strategy-forced generation every builder clears the checklist. Generation *reliability* (3/8, 7/8, 8/8 strategies accepted) — not population behavioral diversity — is where builder capability enters.

6.5 SECOND TARGET: THE PHENOMENA TRANSFER

Re-evaluating {bare, react, A6, D6, human control} with **Qwen3.8-Flash as the frozen target** (151 items, fresh cache): collapse replicates (react vs bare: 0.0% disagreement; A6 mean disagreement 6.8%), and D6 still meets the checklist (union repair 17.1%, headroom 5.96 pp, 6 pairwise-distinct fix sets) — behavioral diversity transfers across targets. One honest counter-result: the human control does not transfer wholesale — `hpc_hint` (hint re-ordering in the prompt) falls to 28.5% accuracy on the Qwen target, while the three *execution-path* controls (repair 55.6%, schema 57.0%, vote3 54.3%) do not exhibit the same collapse. Prompt-side mechanisms are not automatically portable across targets in our one tested case; the control-flow-side variants we tested are.

6.6 DOES DIVERSITY BUY LEARNABLE ROUTING? (HONEST NEGATIVE)

For the two admitted populations we ran the pre-specified leave-one-harness-out (LOHO) value-prediction protocol: logistic heads on task TF-IDF text plus harness code and structural features, one held-out harness per fold, with a hash-based item split (20% tune / 80% eval) used only for threshold selection. It does **not** beat the best fixed harness: D6 −3.4 pp, C6 +0.9 pp (bootstrap CI [−5.2, +6.9]), cross-protocol pool −0.9 pp (CI [−6.0, +4.3]); full per-fold results are in the released JSONs. Two caveats sharpen — rather than soften — this negative. First, LOHO holds the task set fixed, so the high per-fold AUROC on repair (0.82–0.98) largely reflects *task memorization*: in a task-held-out control on D6 (train on 76 tasks, test on the 75 unseen tasks; `task_split_D6.json`), the repair classifier is at chance (mean AUROC 0.512) and the policy degrades to the bare baseline. This is a single-population control, not a sweep; but it removes the main alternative reading of the LOHO AUROC, and task-side generalization of harness value fails where tested. Second, the pre-specified three-level gate returns: L1 (headroom) **passes**; L2 (closed-set, harness identity visible) **marginally fails** — its pre-specified criterion was a policy gain of ≥ 2 pp, which no closed-set configuration demonstrates (closed-set repair AUROC 0.471 on D6, 0.575 on the 24-harness pool); L3 (open-set, unseen harness) **fails** with unseen-harness representation adding ≤ 0.001 AUROC over task-only features. One deployment-relevant positive survives, and we state its scope precisely: on the 24-harness cross-protocol pool only, the same predictor used as an *abstaining gate* gains **+6.9 pp** over bare with zero harness-induced harm on its interventions (the admitted single-builder populations show intervention harm of 2.8% and 8.7%). The open problem stands: estimating an unseen harness’s per-item value from code and task text alone, on tasks the predictor has not seen, is unsolved by our strongest attempt.

7 DISCUSSION

7.1 DISCUSSION

The checklist as a community tool. The three metrics of Section 4 are computable from the outcome matrix any evaluation already produces. A routing paper whose population fails the checklist has, at best, rediscovered its best fixed harness; we argue the checklist should be reported alongside accuracy, the way calibration plots accompany confidence-based methods. **Implications for**

AI4AI generation. T1–T3 imply concrete protocol requirements: (i) acceptance must be defined on *traces*, not docstrings or code diffs; (ii) feedback mechanisms need a demonstrated causal path; (iii) executable-output validation precedes any capability claim. The IR arm shows the third requirement is not enough on its own: compiled mechanism diversity yielded the most distinct fix sets (6/6) yet only 3.97 pp headroom — mechanisms must also be *selected against a headroom landscape*, which is exactly the information the checklist surfaces. **Why stronger builders are not automatically the fix.** Under the gated protocol, the builder axis orders generation reliability and population quality (GLM 3/8 strategies accepted under logging; Qwen 5.96 pp; DeepSeek 7.95 pp). But the old protocol’s failure mode is different in kind: its best member (0.583, the strongest harness in the study) *dominates*, so the population’s remaining headroom falls to 4.64 pp even with a healthy repair rate — missing the threshold by 0.36 pp rather than collapsing. A strong builder without a behavioral gate concentrates quality; a gate without a capable builder produces nothing. The checklist detects which axis is binding — while we acknowledge the design cannot separate main effects. **Limitations.** One domain family (text-to-SQL; a second database for build–eval disjointness), a second frozen target replicating the key phenomena, K=6–8 populations, and execution-accuracy judging (set-of-rows equality, not BIRD’s official scorer). 24 of the 151 evaluation items overlap with earlier protocol-development runs — this overlap is disclosed, and the A-arm population selection used only the 36 fully disjoint items. The human control was designed after inspecting day-1 traces, so its headroom is an existence proof, not a blind estimate. Pre-registered thresholds were set from 60-item evidence; the borderline populations (IR at 3.97 pp, human control at 14.9% repair) are reported exactly as measured, without post-hoc adjustment. Bootstrap intervals on 151 items are wide — roughly ± 9 –11 pp on repair rates — and all raw metrics are released so stricter readings can be applied. Finally, the admission decision follows the frozen point-estimate rule; readers applying interval-based standards would admit fewer populations, and we say so where it matters.

8 CONCLUSION

8.1 CONCLUSION

AI-generated harness populations can be exactly what their source files suggest and nothing like what routing needs. We measured that gap — mean code similarity 0.14 against 96% behavioral agreement — explained its mechanism at the trace level, showed the missing headroom is recoverable by hand (7.3 pp), and then showed it is also reachable automatically: strategy-forced generation with capable builders restores behavioral diversity (two admitted populations, up to 8.0 pp headroom at the point estimate) where the old protocol misses the pre-specified headroom threshold. What diversity does not yet buy is open-set routing — unseen-harness value prediction remains unsolved, under both harness-level and task-level generalization — so the practical output of this paper is deliberately small and reusable: report disagreement, union repair, and headroom before you route; gate generation on traces, not text.

REFERENCES

- Agentic Routing authors. Agentic routing: Routing models within a harness. *arXiv preprint arXiv:2607.11399*, 2026. ID to be re-verified at camera-ready.
- CF-Routing authors. Counterfactual routing over {No-Search, Search, Abstain}. *arXiv preprint arXiv:2607.05752*, 2026. ID to be re-verified at camera-ready.
- et al. Chen. HarnessFix: Trace-grounded diagnosis-driven repair of agent harnesses. *arXiv preprint*, 2026, 2026. Complete ID to be resolved via citation audit.
- Gated-QD authors. Self-evolving agent harnesses via gated semantic quality-diversity. *arXiv preprint arXiv:2607.13683*, 2026.
- GRASP authors. GRASP: Gated skill admission at development time. *arXiv preprint arXiv:2605.29668*, 2026. ID to be re-verified at camera-ready.
- Harness Handbook authors. Harness handbook: Making evolving agent harnesses readable, navigable, and editable. *arXiv preprint arXiv:2607.13285*, 2026.

HarnessLens authors. Verify smarter, evolve further: Efficient harness evolution through behavior-aware verification (harnesslens). *arXiv preprint arXiv:2608.27311*, 2026. ID to be re-verified at camera-ready.

HELIX authors. HELIX: Portfolio harnesses and coverage. *arXiv preprint arXiv:2608.13951*, 2026.

HSNM authors. Harness sensitivity is non-monotone. *arXiv preprint arXiv:2605.26731*, 2026. ID to be re-verified at camera-ready.

JIT-Agent authors. JIT-agent: Just-in-time harness generation. *arXiv preprint arXiv:2608.25593*, 2026. ID to be re-verified at camera-ready.

Jiahang Lin, Shichun Liu, Tao Gui, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint*, 2026, 2026. Complete ID to be resolved via citation audit.

NUSH authors. No universally superior harness. *arXiv preprint arXiv:2607.18235*, 2026. ID to be re-verified at camera-ready.

STS authors. STS: Scaffold selection over a closed set of scaffolds. *arXiv preprint arXiv:2604.06753*, 2026. ID to be re-verified at camera-ready.

Task-CoEvolve authors. Task-coevolve: Co-evolution of tasks and scaffolds. *arXiv preprint arXiv:2608.20169*, 2026. ID to be re-verified at camera-ready.

TTHE authors. TTHE: Test-time harness evolution for text-to-SQL. *arXiv preprint arXiv:2607.08124*, 2026.

et al. Zhang. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026.

A REPRODUCIBILITY AND AUDIT TRAIL

Decision log. The admission thresholds used in Section 6 were recorded in the project decision log before any 151-item outcome existed: the v6 entry (2026-08-31) set a 20% union-repair threshold; the v7 entry (2026-09-01), which fixed the evaluation set at 151 items, records “union repair $\geq 15-20\%$ ” while the same-day experiment plan specifies $\geq 15\%$. We operationalize $\geq 15\%$ and disclose that both readings admit the same populations (C6 at 21.6%, D6 at 24.3%) and leave all other admission decisions unchanged. The repository history is initialized retrospectively (2026-09-03); file mtimes and the decision-log changelog are the authoritative ordering record.

Data hygiene. 24 of 151 evaluation items overlap day-1 protocol-development runs; the A-arm population selection used only the 36 fully disjoint items. The bare baseline flipped 3 of 60 items between the two day-1 runs (API nondeterminism at temperature 0), so 60-item columns are conditioned on slightly different bare-error sets.

Generation provenance. The 09-01 free-form populations (conditions C, D) were produced in-session; their generation tool was not preserved. The tool of record from 2026-09-03 (`builder.generate3.py`) logs every attempt (3 retries, 16,384-token budget, per-attempt stage and finish reason) and accepts a candidate only after it imports against both databases and solves one builder-visible question. The GLM condition-B population reported in Section 6 is the logged re-run.

B POPULATION CONDITIONS (DETAIL)

This appendix details the populations compared in Section 3.3; each AI-generated population contains $K = 6$ harnesses produced under a fixed budget and acceptance protocol unless noted.

- **Seed baselines:** `bare` (single call) and `react` (execution feedback loop, shipped with TTHE).

- **Condition A — proposer protocol (old):** six candidates from TTHER’s own optimize loop (GLM-5.3-Flash as builder), which sees 3 questions per round and edits harness code in a single turn. We select the 6 most behaviorally diverse of 12 candidates using *only* the 36 earlier-run questions disjoint from D_{eval} (selection rule frozen before the 151-item evaluation; the 24 overlapping items were excluded from selection).
- **Condition B — strategy-forced free-form, GLM builder:** the original 09-01 run recorded zero accepted outputs, but its generation tool was not preserved (see Section 6), so that observation is unaudited; under the logging tool of record, GLM accepts **3 of 8 strategies** (execute-repair, 3-vote, two-view) and fails the other five within the retry budget, with per-attempt logs retained. We use the audited re-run (B3, K=3) as the condition-B population and report the unaudited 0/6 only as a provenance caveat.
- **Condition C — strategy-forced free-form, Qwen builder:** Qwen3.8-Flash, 6/6 accepted.
- **Condition D — strategy-forced free-form, DeepSeek builder:** DeepSeek-V4-Flash (vision-exp checkpoint), 6/6 accepted.
- **IR arm — compiled, no free-form code:** six harnesses compiled deterministically from declarative mechanism specifications by our Harness IR (Section 6.2); mechanism diversity is guaranteed by construction, isolating “what mechanisms do” from “can a builder write code”.
- **Human positive control:** four hand-written harnesses with forcibly distinct execution paths (execute-and-repair, 3-vote, schema-linking, hint-first).

C DESIGN AND PRE-REGISTRATION (DETAIL)

This appendix details the design structure of Section 3.4. **The design is two nested contrasts, originally two nested contrasts (the old-protocol row was later completed for Qwen and DeepSeek; Section 6.2),** and our claims are limited accordingly: (i) *within the old proposer protocol*, the day-2 design ran GLM only, and *within the strategy-forced free-form protocol*, only the builder varies (GLM/Qwen/DeepSeek) — so the protocol contrast at a fixed builder exists only for GLM (A vs B), and the builder contrast exists only under the gated protocol (B vs C vs D); (ii) the IR arm has a protocol axis but no builder axis. Protocol and builder effects are therefore partially confounded across conditions; Section 6 interprets each contrast within its own fixed factor and does not claim to separate main effects.

Audit trail. Before running any 151-item evaluation, admission thresholds were frozen in the project decision log: a population must reach union repair rate over bare errors $\geq 15\%$, oracle headroom (vs. best fixed harness) ≥ 5 pp, and ≥ 3 harnesses with non-empty, pairwise-distinct repair sets. The 20% repair threshold first appears in the v6 decision-log entry (2026-08-31, when the plan moved to 150-item evaluation); it was set to 15% in the v7 entry (2026-09-01, when the evaluation set was fixed at 151 items) — *both before any 151-item outcome existed*; the matrix was collected 2026-09-02. The repository now carries a git history initialized from file mtimes and the decision-log changelog, which together document this ordering; we flag that the history is retrospective and the changelog is the authoritative ordering record. Populations failing the thresholds are not subjected to downstream value-prediction experiments; admission is decided on point estimates by the frozen rule, bootstrap intervals are reported alongside for calibration, and Section 6 marks populations whose intervals include sub-threshold values.

D GENERATION AND IR SPECIFICATIONS

Strategy-forced free-form protocol (detail). For each of six mechanism strategies s (execute-repair, 3-vote, schema-linking, hint-guard, decomposition, format-guard), the builder receives a fixed prompt: the `SQLHarness` interface (constructor-provided schema; `self.llm` and `self.execute` primitives; `bridge.extract_sql`), the strategy description (one sentence, e.g. “Generate SQL, then EXECUTE it via `self.execute()`; if execution fails, feed the exact `SQLite` error back and regenerate up to 2 times”), and the required class name. The builder emits one complete Python file per strategy (temperature 0.7); the candidate is extracted from its code fence

and written to the population directory. Acceptance for the 09-01 populations (conditions C and D) was: file written and hash-registered, with import and smoke checks performed at evaluation time; per-attempt generation logs were not retained — a provenance gap disclosed in Section 6. The day-3 generation tool (`builder_generate3.py`) closes this gap for all runs after 2026-09-03 by logging every attempt (3 retries, 16,384-token budget) and accepting a candidate only after it imports against both databases and solves one builder-visible question. Budget: one generation call per strategy per run (16,384 max tokens); failed extractions are not retried within a run — a design choice that keeps per-strategy acceptance an auditable, single-shot reliability observation (condition B’s original unaudited run is superseded by the logged re-run, Section 6.0). The full prompt template, per-strategy descriptions, and the six per-builder acceptance logs are released verbatim.

Harness IR. A spec is a typed record: `context` $\in$ `{plain, schema_link, hint_guard, link+hint}`, `generate.candidates` ≥ 1 with `temperature`, `execution.execute_each` $\in$ `{0, 1}`, `repair.{on_error, max_rounds}`, `selection` $\in$ `{first_ok, majority_rows, llm_judge}`. The compiler emits deterministic Python from fixed templates; the builder (or human) chooses and parameterizes modules but cannot emit control flow, so a declared mechanism is present in the control flow *by construction*. The six IR harnesses enumerate the spec grid (plain/repair/vote3/hint-repair/schema-repair/link+vote3).

E ADDITIONAL RESULTS

Per-harness accuracies, per-fold LOHO tables, bootstrap intervals, fix-set audits, and the task-held-out control (`task_split_D6.json`) are included in the anonymous code release.